Spring: A Head Start 🔥 — Spring Events (Part 3)

Event Handling and Custom Events.

Omar Elgabry Follow
Sep 12, 2017 · 2 min read

Spring: A Head Start 🔥 — Spring Events

Spring Events

An event in Spring is a class that extends *ApplicationEvent* class, published by a class that implements *ApplicationEventPublisherAware*, and handled by a class that implements *ApplicationListener* interface.

ApplicationContext Events

The *ApplicationContext* manages the complete life cycle of the beans. *Sounds familiar, Isn't it?*.

In Spring, we have a set of pre-defined events that get published by the *ApplicationContext* during life cycle of the context.

For example, a *ContextStartedEvent* is published when the context is started and *ContextStoppedEvent* is published when the context is stopped.

> 💡*This is useful so we can hook into the life cycle of the context and add in our own custom logic.*

So, If a class implements the *ApplicationListener*, then every time an event gets published by the *ApplicationContext*, that class is notified.

> 💡*Spring's event are synchronous (single-threaded). So, if an event is published, until and unless all the receivers get the message, the processes are blocked and the flow will not continue.*

Listening To ApplicationContext Events 🎧

To listen to a context event, a class (listener) should implement the *ApplicationListener* interface and override the `onApplicationEvent()` method.

```
import org.springframework.context.ApplicationListener;
import org.springframework.context.event.ContextStartedEvent;

public class ContextStartEventHandler implements ApplicationListener<ContextStartedEvent>{

        @Override
        public void onApplicationEvent(ContextStartedEvent event) {
                System.out.println("The ApplicationContext is started on the ConfigurableApplicationC
        }
}
```

ContextStartEventHandler.java hosted with by GitHub view raw

```java
import org.springframework.context.ApplicationContext;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {
  public static void main(String[] args) {
    // ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");
    // fire the start event.
    // ((ConfigurableApplicationContext) context).start();

    ConfigurableApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");
    // fire the start event.
    context.start();
  }
}
```

> *You also need to declare this class as a bean (i.e. In XML configuration file) so that the container can identify the bean as an event listener, thus will be notified.*

```xml
<bean id="contextStartEventHandler" class="ContextStartEventHandler">
</bean>
```

Custom Events

Instead, you can write, publish and handle your own custom Spring events. All what you need to do is as the following:

- Create the **event**, and extend *ApplicationEvent* class.
- Create the **publisher**, and implement *ApplicationEventPublisher (or ApplicationEventPublisherAware)*.
- Create the **listener**, and implement *ApplicationListener*.

The Event

The event class is to store the event data. Let's create a simple event class.

```java
import org.springframework.context.ApplicationEvent;

public class CustomEvent extends ApplicationEvent {

    public CustomEvent(Object source, String message) {
        super(source);
    }

    public String toString() {
        return "This is a custom event";
    }
}
```

CustomEvent.java hosted with ♡ by GitHub view raw

The Publisher

The publisher constructs the event object and publishes it to anyone who's listening.

To publish an event, either:

[1] Inject the *ApplicationEventPublisher* object using `@Autowired` annotation.

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationEventPublisher;

public class CustomEventPublisher {
  @Autowired
  private ApplicationEventPublisher publisher;

  public void publish() {
    CustomEvent event = new CustomEvent(this);
    publisher.publishEvent(event);
  }
}
```

CustomEventPublisher.java hosted with ♡ by GitHub view raw

[2] The publisher class can implement the *ApplicationEventPublisherAware* interface, and the `setApplicationEventPublisher()` method which sets the *ApplicationEventPublisher object*.

```java
import org.springframework.context.ApplicationEventPublisher;
```

```
import org.springframework.context.ApplicationEventPublisherAware;

public class CustomEventPublisher implements ApplicationEventPublisherAware {
  private ApplicationEventPublisher publisher;

  public void setApplicationEventPublisher (ApplicationEventPublisher publisher){
    this.publisher = publisher;
  }

  public void publish() {
    CustomEvent event = new CustomEvent(this);
    publisher.publishEvent(event);
  }
}
```

CustomEventPublisher.java hosted with ♡ by GitHub view raw

> ⚠ *In case of publisher implements the ApplicationEventPublisherAware interface. The publisher class has to be declared as a bean so that the container can identify the bean as an event publisher.*

```
<bean id="customEventPublisher" class="CustomEventPublisher"/>
```

The Listener/Handler

The listener fires whenever the event gets published.

It implements *ApplicationListener* interface and has to be defined as a bean (as we did before).

```
<bean id="customEventHandler" class="CustomEventHandler"/>
```

beans.xml hosted with ♡ by GitHub view raw

```
import org.springframework.context.ApplicationListener;

public class CustomEventHandler implements ApplicationListener<CustomEvent> {
  public void onApplicationEvent(CustomEvent event) {
    System.out.println("Printing Event " + event.toString());
  }
}
```

CustomEventHandler.java hosted with ♡ by GitHub view raw

Putting All Together

Let's test our custom event.

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {
  public static void main(String[] args) {
    ApplicationContext context = new ClassPathXmlApplicationContext("Beans.xml");

    CustomEventPublisher publisher = (CustomEventPublisher) context.getBean("customEventPublisher");
    publisher.publish();
  }
}
```

Test.java hosted with ♡ by GitHub view raw

Thank you for reading! If you enjoyed it, please clap 👏 for it.

Java Spring Framework Programming Software Development Other

About Help Legal

Get the Medium app

Download on the App Store GET IT ON Google Play